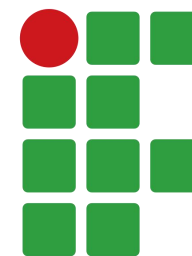


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

Orientação a Objetos em Kotlin

- Definindo uma classe simples e instanciando um objeto em Kotlin:

```
class Pessoa {  
    var nome: String = ""  
    var idade: Int = 0  
  
    fun saudacao() {  
        println("Olá, meu nome é $nome e eu tenho $idade anos.")  
    }  
}
```

```
fun main() {  
    val pessoa = Pessoa()  
    pessoa.nome = "Lucas"  
    pessoa.idade = 18  
    pessoa.saudacao()  
}
```

Orientação a Objetos em Kotlin

- Construtores primários: Kotlin permite definir o construtor diretamente no cabeçalho da classe.

```
class Pessoa(val nome: String, var idade: Int){  
    override fun toString() = "Pessoa[nome = $nome, idade = $idade]"  
}  
  
fun main() {  
    val pessoa = Pessoa("Alice", 30)  
    println(pessoa)  
}
```



```
Pessoa[nome = Alice, idade = 30]
```

Orientação a Objetos em Kotlin

- Construtores primários: Kotlin permite definir o construtor diretamente no cabeçalho da classe.

```
class Pessoa(val nome: String, var idade: Int){  
    override fun toString() = "Pessoa[nome = $nome, idade = $idade]"  
}
```

Propriedade pode ser modificada.

Propriedade não pode receber uma nova referência após inicializada.

```
println(pessoa)
```



```
Pessoa[nome = Alice, idade = 30]
```

Orientação a Objetos em Kotlin

- Kotlin fornece automaticamente getters e setters para propriedades.
 - `var`: possui getter e setter.
 - `val`: possui apenas getter, pois não pode ser alterado depois da inicialização.

```
class Produto(  
    var nome: String,  
    var preco: Double  
)
```

```
fun main(){  
    val produto = Produto("Mouse", 50.0)  
  
    println(produto.nome)  
  
    produto.preco = 45.0  
}
```

Orientação a Objetos em Kotlin

- O bloco `init`: é executado durante a criação do objeto e pode ser usado para inicializar propriedades, realizar validações ou executar outras instruções de inicialização.

```
class Pessoa(  
    val nome: String,  
    val idade: Int  
) {  
    init {  
        println("Pessoa criada: $nome")  
    }  
}
```

Orientação a Objetos em Kotlin

- O bloco `init`: é executado durante a criação do objeto e pode ser usado para inicializar propriedades, realizar validações ou executar outras instruções de inicialização.

```
class Pessoa(  
    val nome: String,  
    val idade: Int  
) {  
    init {  
        require(idade >= 0) {  
            "A idade não pode ser negativa."  
        }  
    }  
}
```

Orientação a Objetos em Kotlin

- Construtores secundários: são definidos dentro do corpo da classe.

```
class Pessoa(  
    val nome: String,  
    val idade: Int  
) {  
    constructor(nome: String) : this(nome, 0)  
}
```


Orientação a Objetos em Kotlin

- Construtores secundários: são definidos dentro do corpo da classe.

```
class Pessoa(  
    val nome: String,  
    val idade: Int  
) {  
    constructor(nome: String) : this(nome, 0)  
}
```

Em Kotlin, um construtor secundário deve obrigatoriamente delegar para o construtor primário, quando a classe possui um construtor primário.

Orientação a Objetos em Kotlin

- Construtores secundários: são definidos dentro do corpo da classe.

```
class Pessoa(  
    val nome: String,  
    val idade: Int,  
    val endereco: String  
) {  
  
    constructor(nome: String, idade: Int)  
        : this(nome, idade, "Não informado")  
  
    constructor(nome: String)  
        : this(nome, 0, "Não informado")  
  
}
```

Orientação a Objetos em Kotlin

- Construtor primário com valor padrão:

```
class Pessoa(val nome: String, var idade: Int = 0,  
             var endereco: String = "Endereço não informado")  
  
fun main() {  
    val pessoa1 = Pessoa("João")  
    val pessoa2 = Pessoa("Maria", 25)  
    val pessoa3 = Pessoa("Carlos", 40, "Rua Nova, 123")  
}
```

Orientação a Objetos em Kotlin

- O construtor primário não possui um corpo próprio para executar instruções.
- A inicialização pode ser realizada diretamente nas propriedades ou em blocos `init`.
- Um construtor secundário deve delegar sua inicialização ao construtor primário, utilizando `this`.

Orientação a Objetos em Kotlin

- Modificadores de acesso:
 - public (padrão)
 - private
 - protected
 - internal

```
class Pessoa {  
    public val nome = "Lucas",  
    private val cpf = "123...",  
    protected val idade = 30  
}
```

Orientação a Objetos em Kotlin

- Modificadores de acesso:

```
class Pessoa(private var idade: Int) {  
  
    fun getIdade(): Int {  
        return idade  
    }  
  
    fun setIdade(idade: Int){  
        this.idade = idade;  
    }  
}
```

Orientação a Objetos em Kotlin

- Modificadores de acesso:

```
class Pessoa(idadeInicial: Int) {  
    var idade: Int = idadeInicial  
    private set  
}
```

getter publico e setter privado

```
fun main(){  
    val pessoa = Pessoa(18)  
    println(pessoa.idade)  
    pessoa.idade = 19  
}
```


Orientação a Objetos em Kotlin

- Modificadores de acesso:

```
class Pessoa(idadeInicial: Int) {  
    var idade: Int = idadeInicial  
    private set  
}  
  
fun main(){  
    val pessoa = Pessoa(18)  
    println(pessoa.idade) ✓  
    pessoa.idade = 19 ✗  
}
```


Orientação a Objetos em Kotlin

- Data classes são indicadas para classes cujo principal objetivo é armazenar dados.
- O Kotlin gera automaticamente várias funcionalidades úteis:
 - equals(): Compara dois objetos para verificar se seus dados são iguais.
 - hashCode(): Gera um código hash baseado nas propriedades dos dados.
 - toString(): Gera uma representação em string dos dados da classe.
 - copy(): Permite criar cópias do objeto com algumas propriedades modificadas.

Orientação a Objetos em Kotlin

```
data class Pessoa(val nome: String, val idade: Int)

fun main() {
    val pessoa1 = Pessoa("João", 30)
    val pessoa2 = Pessoa("João", 30)

    println(pessoa1) // toString: Pessoa(nome=João, idade=30)
    println(pessoa1 == pessoa2) // equals: true
    println(pessoa1.hashCode() == pessoa2.hashCode()) // hashCode: true

    val pessoa3 = pessoa1.copy(idade = 31)
    println(pessoa3) // Pessoa(nome=João, idade=31)
}
```

Orientação a Objetos em Kotlin

- Companion Objects: É utilizado para definir membros associados à classe, com comportamento semelhante aos membros static do Java.

```
class Constants {  
    companion object {  
        const val PI = 3.14159  
        const val DEFAULT_TIMEOUT = 5000  
        val VERSION_NAME = "1.0.0"  
    }  
}  
  
fun main() {  
    val piValue = Constants.PI  
    val timeoutValue = Constants.DEFAULT_TIMEOUT  
    val versionName = Constants.VERSION_NAME  
}
```

Orientação a Objetos em Kotlin

- Companion Objects: É utilizado para definir membros associados à classe, com comportamento semelhante aos membros static do Java.

```
class Matematica {  
    companion object {  
        fun dobro(valor: Int): Int {  
            return valor * 2  
        }  
    }  
}  
  
fun main(){  
    println(Matematica.dobro(10))  
}
```

Orientação a Objetos em Kotlin

- Herança: por padrão, classes Kotlin são finais (final). Para permitir herança, usamos open.

```
open class Animal {  
    fun comer() {  
        println("Animal comendo")  
    }  
}  
  
class Cachorro : Animal() {  
    fun latir() {  
        println("Au au!")  
    }  
}
```

```
fun main(){  
    val cachorro = Cachorro()  
  
    cachorro.comer()  
    cachorro.latir()  
}
```

Orientação a Objetos em Kotlin

- Sobrescrita (override): uma classe filha pode modificar o comportamento herdado.

```
open class Animal {  
    open fun emitirSom() {  
        println("Som")  
    }  
}  
  
class Cachorro : Animal() {  
    override fun emitirSom() {  
        println("Au au!")  
    }  
}
```

```
fun main(){  
    val animal: Animal = Cachorro()  
  
    animal.emitirSom()  
}
```


Orientação a Objetos em Kotlin

- Polimorfismo: uma referência de uma classe mais geral pode apontar para objetos de classes especializadas.

```
open class Animal {  
    open fun emitirSom() {  
        println("Som")  
    }  
}  
  
class Cachorro : Animal() {  
    override fun emitirSom() {  
        println("Au au!")  
    }  
}  
  
class Gato : Animal() {  
    override fun emitirSom() {  
        println("Miau!")  
    }  
}
```


Orientação a Objetos em Kotlin

- Polimorfismo: uma referência de uma classe mais geral pode apontar para objetos de classes especializadas.

```
fun main(){  
    val animais: List<Animal> = listOf(  
        Cachorro(),  
        Gato()  
    )  
  
    animais.forEach {  
        it.emitirSom()  
    }  
}
```

```
open class Animal {  
    open fun emitirSom() {  
        println("Som")  
    }  
}  
  
class Cachorro : Animal() {  
    override fun emitirSom() {  
        println("Au au!")  
    }  
}  
  
class Gato : Animal() {  
    override fun emitirSom() {  
        println("Miau!")  
    }  
}
```

Orientação a Objetos em Kotlin

- `super`: permite acessar a implementação da classe pai.

```
open class Animal {  
    open fun emitirSom() {  
        println("Som do animal")  
    }  
}  
  
class Cachorro : Animal() {  
    override fun emitirSom() {  
        super.emitirSom()  
        println("Au au!")  
    }  
}
```

Orientação a Objetos em Kotlin

- Classe abstrata: serve como uma base que não pode ser instanciada diretamente.

```
abstract class Animal {  
    abstract fun emitirSom()  
  
    fun dormir() {  
        println("Dormindo...")  
    }  
}  
  
class Cachorro : Animal() {  
    override fun emitirSom() {  
        println("Au au!")  
    }  
}
```

Orientação a Objetos em Kotlin

- Classe abstrata: serve como uma base que não pode ser instanciada diretamente.

```
abstract class Animal {  
    abstract fun emitirSom()  
  
    fun dormir() {  
        println("Dormindo...")  
    }  
}  
  
class Cachorro : Animal() {  
    override fun emitirSom() {  
        println("Au au!")  
    }  
}
```

```
val animal = Animal()
```



```
val cachorro = Cachorro()
```



Orientação a Objetos em Kotlin

- Interfaces: definem comportamentos que uma classe pode implementar.

```
interface Voador {  
    fun voar()  
}  
  
class Passaro : Voador {  
    override fun voar() {  
        println("O pássaro está voando")  
    }  
}
```

Orientação a Objetos em Kotlin

- Interfaces: uma classe pode implementar várias interfaces.

```
interface Voador {  
    fun voar()  
}  
  
interface Nadador {  
    fun nadar()  
}
```

```
class Pato : Voador, Nadador {  
    override fun voar() {  
        println("Voando")  
    }  
  
    override fun nadar() {  
        println("Nadando")  
    }  
}
```


Orientação a Objetos em Kotlin

- Interfaces: Uma interface pode possuir implementação padrão. A classe que implementar a interface pode utilizar o comportamento padrão ou sobrescrevê-lo.

```
interface Animal {  
    fun emitirSom()  
  
    fun dormir() {  
        println("Dormindo...")  
    }  
}
```


Exercício rápido

1. Crie uma classe Conta contendo:

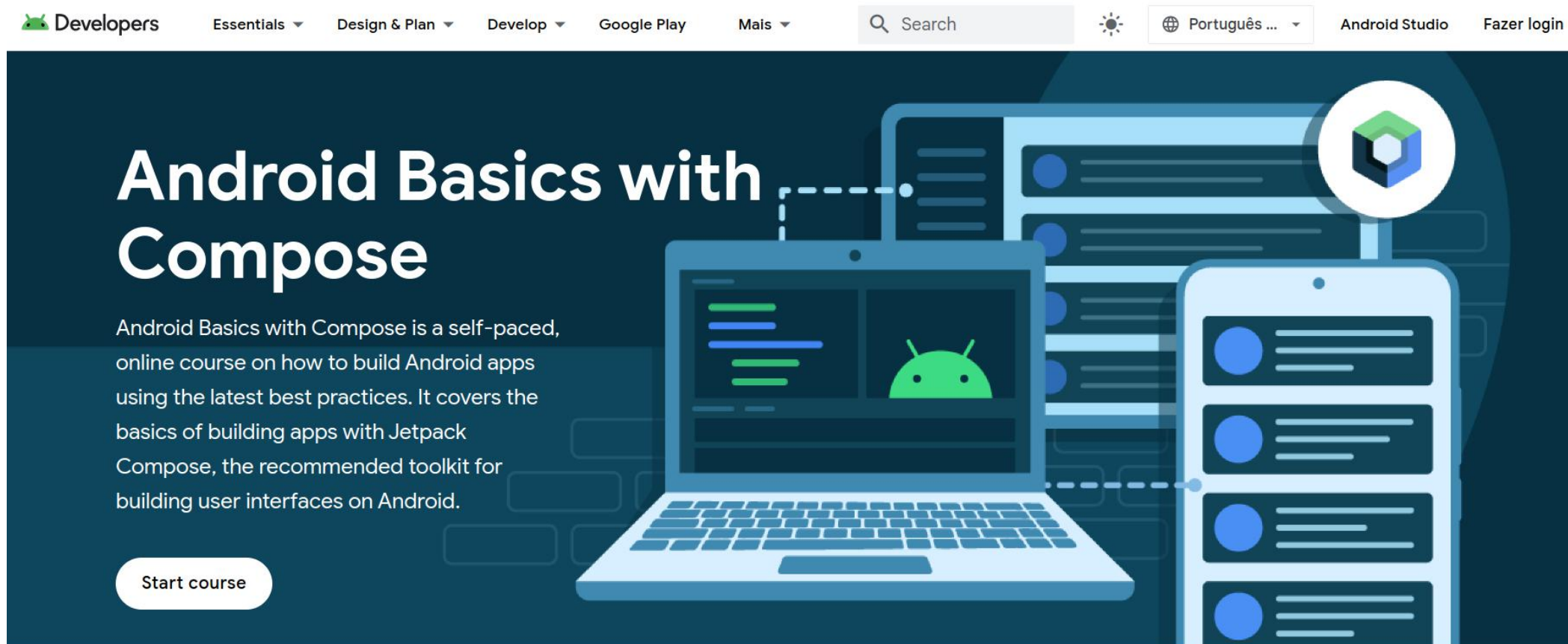
- número da conta;
- titular;
- saldo.
 - O saldo não deve poder ser alterado diretamente fora da classe.

2. Implemente os métodos:

- depositar(valor)
 - O valor depositado deve ser positivo.
- sacar(valor)
 - O saque só deve ser permitido quando o valor for positivo e houver saldo suficiente.
- consultarSaldo()

3. No main, crie uma conta e realize algumas operações.

Prática de laboratório



The screenshot shows the top navigation bar of the Android Developers website. It includes the 'Developers' logo, a menu with 'Essentials', 'Design & Plan', 'Develop', 'Google Play', and 'Mais', a search bar, a language selector set to 'Português ...', and links for 'Android Studio' and 'Fazer login'. The main content area features a dark blue background with the title 'Android Basics with Compose' in large white text. Below the title is a paragraph describing the course as a self-paced, online course on building Android apps with Jetpack Compose. A 'Start course' button is located at the bottom left of the main content area. The background illustration includes a laptop displaying code and the Android robot, a smartphone, and server racks, all connected by dashed lines.

Android Basics with Compose

Android Basics with Compose is a self-paced, online course on how to build Android apps using the latest best practices. It covers the basics of building apps with Jetpack Compose, the recommended toolkit for building user interfaces on Android.

[Start course](#)

Link: <https://developer.android.com/courses/android-basics-compose/course>

Prática de laboratório

MÓDULO 2



5 ATIVIDADES

Configurar o Android Studio

Instale e configure o Android Studio. Crie e execute seu primeiro projeto em um dispositivo ou emulador.

Abril de 2022

Confira

Link: <https://developer.android.com/courses/android-basics-compose/course>

Lista de nivelamento disponível

- Prazo de entrega: 23/08/2026 às 23h59.

 **ACESSAR**

INSIRA A CHAVE PARA ACESSAR A DISCIPLINA

INFORMAÇÃO DA DISCIPLINA

Por favor insira o ID e a chave fornecida para acessar a disciplina.

ID DISCIPLINA

CHAVE

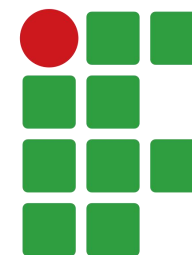
ENTRAR

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco